

COMPLEXITY-DEEP : Routage lexical résiduel déterministe pour les MLP de modèles de langage

Anonymous authors

Paper under double-blind review

Abstract

Nous présentons COMPLEXITY-DEEP, un modèle decoder-only qui ajoute à un tronc MLP dense partagé une petite branche lexicale résiduelle déterministe. Une assignation primaire modulo permutée et une assignation secondaire déterministe équilibrée sélectionnent deux experts par token. Dans une comparaison appariée à graine unique, environ 300M paramètres et 8B tokens FineWeb-Edu, le dernier checkpoint d'évaluation commun (step 7 500) atteint 2,932897 contre 2,948246 pour dense sur un flux fixe issu du split d'entraînement. Au step 7 620, les pertes train sont 2,923100 et 2,932437 ; la moyenne des 50 derniers steps uniques est 2,928258 contre 2,944597. L'affirmation est volontairement limitée : cette paire de runs montre un gain à budget de tokens égal, sans établir un avantage wall-clock, un résultat held-out indépendant ou un routage sensible aux fréquences.

1 Introduction

Les grands modèles de langage (LLM) ont révolutionné le traitement du langage naturel ces dernières années (Vaswani et al., 2017; Brown et al., 2020). Cependant, les architectures dominantes présentent plusieurs limitations :

- **Coût computationnel des MoE** : les architectures Mixture of Experts (Shazeer et al., 2017; Fedus et al., 2022) nécessitent des mécanismes complexes d'équilibrage de charge et de routage.
- **Instabilité de l'entraînement** : les grands modèles souffrent souvent d'instabilités numériques nécessitant un réglage minutieux des hyperparamètres.

Nous proposons COMPLEXITY-DEEP, une famille d'architectures centrée sur le routage lexical déterministe :

1. **Routage lexical fixe** : les IDs de tokens sont mappés vers des paires d'experts par des permutations déterministes de partitions modulo.
2. **MLP résiduel shared-plus-routed** : un grand MLP dense traite tous les tokens et de petits experts routés ajoutent une capacité conditionnée par l'identité lexicale.

La branche routée doit être vue comme un chemin lexical résiduel au-dessus d'un MLP dense partagé, et non comme un remplacement du calcul dense. Les résultats concernent l'architecture combinée et ne suffisent pas à établir une spécialisation sémantique des experts.

2 Travaux connexes

2.1 Architectures Transformer

L'architecture Transformer (Vaswani et al., 2017) est devenue le standard pour les modèles de langage. Les développements récents incluent les optimisations de l'attention comme Flash Attention (Dao et al., 2022),

Grouped Query Attention (GQA) (Ainslie et al., 2023) et Rotary Position Embeddings (RoPE) (Su et al., 2021).

2.2 Mixture of Experts

Les architectures MoE (Shazeer et al., 2017) permettent d’augmenter la capacité sans accroître proportionnellement le calcul. Switch Transformer (Fedus et al., 2022) et Mixtral (Jiang et al., 2024) ont démontré l’efficacité de cette approche, au prix d’une complexité accrue (équilibrage de charge, communication inter-GPU).

2.3 Routage lexical fixe et experts partagés

Le précédent le plus proche du routage par identité de token est Hash Layers (Roller et al., 2021), qui sélectionne des paramètres feed-forward par hash fixe, sans paramètres de routeur ni perte auxiliaire d’équilibrage. COMPLEXITY-DEEP conserve au contraire un grand chemin dense pour chaque token et n’utilise le lookup fixe que pour une branche résiduelle top-2 étroite. DeepSeekMoE utilise également des experts partagés pour capturer des connaissances communes et limiter la redondance entre experts routés (Dai et al., 2024). Une analyse statistique récente obtient, sous des hypothèses explicites d’estimation, des résultats conditionnels d’efficacité en échantillons pour les shared experts (Nguyen et al., 2025). Ces garanties ne s’appliquent pas directement à notre lookup lexical fixe ; elles motivent la décomposition shared-plus-routed mais ne constituent pas un théorème sur le checkpoint évalué.

2.4 Normalisation et stabilité

RMSNorm (Zhang & Sennrich, 2019) et QK-Normalization (Dehghani et al., 2023) sont des techniques modernes pour stabiliser l’entraînement des grands modèles.

3 Architecture COMPLEXITY-DEEP

3.1 Vue d’ensemble

COMPLEXITY-DEEP est une architecture de type décodeur uniquement composée de L couches identiques. Chaque couche comprend :

1. Un bloc d’attention multi-têtes
2. Un bloc MLP avec routage par token
3. Connexions résiduelles avec pré-normalisation (RMSNorm)

La dimension cachée est d_{model} , avec n_h têtes d’attention et n_{kv} têtes clé/valeur (GQA). La Figure 1 donne un schéma simplifié du design Token-Routed résiduel 300M utilisé dans la comparaison de scaling corrigée.

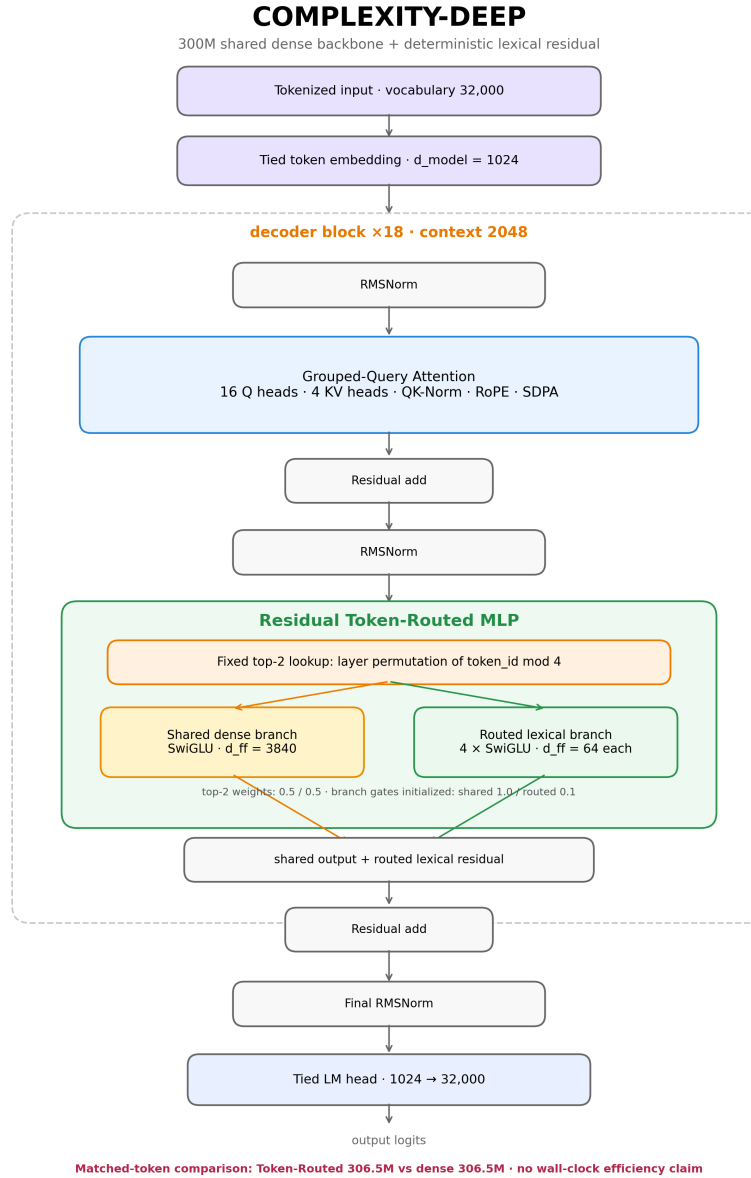


Figure 1: Schéma simplifié de l’architecture COMPLEXITY-DEEP 300M. Chaque couche comprend une attention GQA, une grande branche MLP dense partagée et une petite branche lexicale résiduelle routée de manière déterministe.

3.2 Token-Routed MLP

Le modèle évalué utilise une table top-2 fixe dépendant uniquement de l'identité du token. Pour la couche l , l'expert primaire est obtenu par une permutation π_l du résidu de l'ID :

$$r_{l,1}(t) = \pi_l(t \bmod E), \quad E = 4. \quad (1)$$

Chaque couche assigne ainsi exactement 8 000 des 32 000 IDs à chaque expert primaire. La table secondaire est construite une fois, par ordre croissant d'ID, en choisissant l'expert distinct du primaire actuellement le moins utilisé :

$$r_{l,2}(t) = \arg \min_{e \neq r_{l,1}(t)} \sum_{u=0}^{t-1} \mathbb{I}[r_{l,2}(u) = e], \quad (2)$$

avec départage déterministe par l'indice le plus faible. Il s'agit de la règle *modulo-primaire/secondaire équilibré* observée dans les checkpoints. Les deux sorties top-2 reçoivent un poids fixe de 0,5 :

$$\text{MLP}_l(\mathbf{x}, t) = g_l^s \text{Shared}_l(\mathbf{x}) + g_l^r \sum_{j=1}^2 0.5 \text{Expert}_{r_{l,j}(t)}(\mathbf{x}), \quad (3)$$

où les gates scalaires apprises g_l^s et g_l^r sont initialisées respectivement à 1,0 et 0,1. Elles sont distinctes des poids fixes 0,5/0,5 combinant les deux experts.

Chaque expert est un MLP standard avec activation SiLU (SwiGLU) :

$$\text{Expert}_i(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}_{gate}^i) \odot \mathbf{x}\mathbf{W}_{up}^i) \mathbf{W}_{down}^i \quad (4)$$

Ce que conditionne le routage lexical. Le routage fixe ne regroupe pas les états cachés par sens. Il assigne chaque identité de token à deux fonctions expertes, qui reçoivent cependant des états contextuels produits par les couches d'attention précédentes. La perte de langage est calculée après le réseau résiduel complet et la tête de sortie ; il n'existe pas d'objectif supervisé indépendant par expert. Des assignations lexicales différentes peuvent induire des distributions d'entraînement différentes, mais elles n'impliquent ni gradients indépendants, ni poids orthogonaux, ni spécialisation sémantique.

Avantages opérationnels :

- Pas de perte auxiliaire d'équilibrage de charge
- Lookup statique sans réseau appris de sélection des experts

3.3 Architecture finale

L'architecture évaluée combine un dispatch sparse, une grande branche dense partagée et une petite branche résiduelle top-2.

3.3.1 Dispatch sparse de la branche routée

L'implémentation n'évalue chaque expert routé que sur les tokens qui lui sont assignés :

$$\text{out}[\text{mask}_e] = \text{Expert}_e(\mathbf{x}[\text{mask}_e]) \quad \text{pour } e = 0, \dots, E - 1 \quad (5)$$

Cela évite d'exécuter tous les experts routés sur tous les tokens, sans rendre le modèle complet shared-plus-top-2 équivalent à $1 \times$ dense en temps réel.

3.3.2 MLP dense partagé

Un MLP dense partagé traite *tous* les tokens. Deux experts étroits fournissent en parallèle une correction lexicale résiduelle ; nous ne prétendons pas que ces experts acquièrent une spécialisation sémantique. La

sortie combine les deux branches :

$$\text{MLP}_{\text{output}}(\mathbf{x}, t) = g_s \text{SwiGLU}_{\text{shared}}(\mathbf{x}) + g_r \sum_{j=1}^2 0.5 \text{SwiGLU}_{r_j(t)}(\mathbf{x}) \quad (6)$$

où chaque composant est un MLP SwiGLU standard :

$$\text{SwiGLU}_{\text{shared}}(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}_{\text{gate}}^s) \odot \mathbf{x}\mathbf{W}_{\text{up}}^s) \mathbf{W}_{\text{down}}^s \quad (7)$$

$$\text{SwiGLU}_e(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}_{\text{gate}}^e) \odot \mathbf{x}\mathbf{W}_{\text{up}}^e) \mathbf{W}_{\text{down}}^e \quad (8)$$

Dans le checkpoint 300M, la largeur intermédiaire partagée est 3 840 ; chacun des quatre experts routés a une largeur 64 et deux sont actifs par token. La branche routée est donc une petite correction du tronc dense.

3.3.3 Assignment du vocabulaire

Les tables sauvegardées confirment un routage modulo permuté propre à chaque couche. Chaque expert primaire reçoit exactement 8 000 des 32 000 IDs. Il s'agit d'un équilibre de cardinalité, pas d'un équilibre de fréquence corpus.

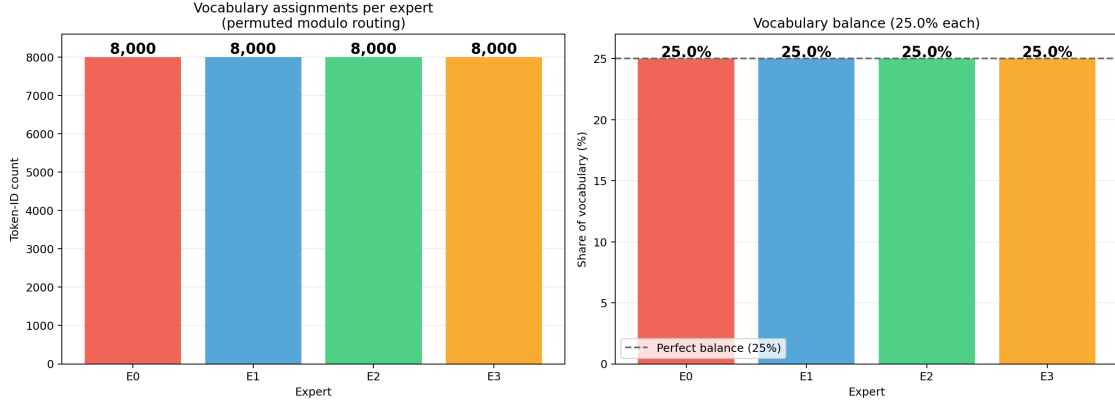


Figure 2: Assignment du vocabulaire extraite du checkpoint 300M : exactement 8 000 IDs sur 32 000 par expert primaire via une permutation modulo propre à chaque couche. La figure ne représente pas le trafic corpus.

4 Analyse théorique

Nous énonçons uniquement des propriétés directement applicables à l'architecture shared-plus-top-2 évaluée.

4.1 Équilibre des assignments primaires

Proposition 4.1 (Équilibre des assignments primaires). *Soient les IDs de tokens $\{0, \dots, |V| - 1\}$ et E experts. Sous $r_{l,1}(t) = \pi_l(t \bmod E)$, chaque expert reçoit soit $\lfloor |V|/E \rfloor$, soit $\lceil |V|/E \rceil$ IDs primaires. Si $E \mid |V|$, chacun en reçoit exactement $|V|/E$.*

Proof. Avant permutation, la classe de résidu e contient les IDs $e, e + E, e + 2E, \dots$ inférieurs à $|V|$. Leurs cardinalités diffèrent d'au plus un. Comme π_l est une bijection des labels d'experts, elle conserve ces cardinalités. Pour $|V| = 32000$ et $E = 4$, chaque couche assigne donc exactement 8 000 IDs primaires à chaque expert. $\square$

Il s'agit d'un équilibre de cardinalité du vocabulaire, et non d'un équilibre du trafic corpus. Des fréquences non uniformes peuvent produire un trafic réalisé inégal.

4.2 Capacité appariée et largeur MLP active

Soient d la dimension du modèle, d_s la largeur du SwiGLU partagé, d_e la largeur de chaque expert routé, E le nombre d’experts stockés et k le nombre sélectionné par token. En ignorant les biais, le nombre de paramètres MLP par couche est

$$P_{\text{TR}} = 3d(d_s + Ed_e), \quad (9)$$

et la largeur évaluée pour chaque token est

$$d_{\text{active}} = d_s + kd_e. \quad (10)$$

Pour le modèle évalué, $(d_s, E, d_e, k) = (3840, 4, 64, 2)$. La largeur MLP stockée vaut donc $3840 + 4 \times 64 = 4096$, exactement comme la baseline dense, tandis que chaque token traverse une largeur $3840 + 2 \times 64 = 3968$. Ce comptage ne garantit aucun gain wall-clock : le dispatch et les petits kernels experts ajoutent un overhead mesuré empiriquement.

4.3 Portée de l’interprétation

Le lookup déterministe supprime le réseau appris de sélection et les pertes auxiliaires d’équilibrage, mais renonce aussi au routage adaptatif au contenu. Les mesures rapportées n’établissent pas de spécialisation sémantique ou fonctionnelle. L’évidence concerne la loss de langage de bout en bout, la structure des tables et le trafic observé.

5 Implémentation

5.1 Spécifications techniques

Notre implémentation utilise PyTorch 2.0+ avec les optimisations suivantes :

Table 1: Choix d’implémentation

Composant	Choix technique
Attention	SDPA / Flash Attention
Encodage positionnel	RoPE ($\theta = 10000$)
Normalisation	RMSNorm + QK-Norm
Activation	SiLU (SwiGLU)
Précision	BF16 (entraînement et inférence)

5.2 Configuration du modèle 300M

Pour la comparaison iso-paramètres corrigée au scale (Section 7.5), nous entraînons deux architectures à $\sim 300\text{M}$ paramètres (Table 2) : un modèle Token-Routed résiduel (306,5M, 4 experts, top- $k = 2$, grand shared expert et petite branche routée) et une baseline dense SwiGLU (306,5M, $d_{ff} = 4096$). Les deux partagent le même backbone (18 couches, $d_{model} = 1024$, GQA 16/4, RMSNorm, QK-Norm), tokenizer, longueur de séquence, optimiseur, warmup et budget FineWeb-Edu.

6 Expériences

6.1 Étude pilote

Des runs pilotes antérieurs ont motivé le retrait du contrôleur résiduel adaptatif et le maintien d’un chemin dense partagé. Les preuves de cet article sont centrées sur la comparaison iso-paramètres 300M corrigée ; les runs pilotes ne soutiennent pas le claim principal.

Table 2: Configurations COMPLEXITY-DEEP 300M (comparaison iso-paramètres)

Paramètre	Token-Routed (MoE)	Baseline dense
Couches (L)	18	18
Dimension (d_{model})	1024	1024
Têtes d’attention (n_h)	16	16
Têtes KV (n_{kv})	4 (ratio GQA 4:1)	4 (ratio GQA 4:1)
Dimension intermédiaire routée	256 total	—
Intermédiaire expert	64	—
Intermédiaire shared expert	3840	—
Experts ($n_{experts}$)	4, top- $k = 2$	1 (dense)
Initialisation gates shared/routed	1,0 / 0,1	—
Vocabulaire	32000	32000
Contexte max	2048	2048
Total paramètres	306,5M	306,5M
Chemin MLP actif/token	shared + 2 experts routés	SwiGLU dense

6.2 Recette d’entraînement

Notre recette d’entraînement suit les pratiques standard des LLMs modernes (LLaMA, GPT-3) avec deux ajustements automatiques :

Warmup dynamique. Au lieu d’un nombre fixe d’étapes de warmup (qui peut représenter 52% du training pour des runs courts), le warmup est automatiquement fixé à **5% du nombre total d’étapes**. Cela garantit un ratio warmup/training constant indépendamment de la durée du run.

Initialisation GPT-style. Les projections de sortie résiduelles (`o_proj`, `down_proj`) sont initialisées avec un écart-type réduit :

$$\sigma_{\text{residual}} = \frac{0.02}{\sqrt{2 \times L}} \quad (11)$$

où L est le nombre de couches. Cela empêche le flux résiduel de croître avec la profondeur (Radford et al., 2019).

Autres hyperparamètres. Weight decay = 0.1 (standard GPT-3/LLaMA), AdamW avec $\beta_1 = 0.9$, $\beta_2 = 0.95$, gradient clipping = 1.0, précision BF16, scheduler cosine avec min_lr = 10% du peak.

6.3 Vérification des tables de routage

Nous avons inspecté le checkpoint final plutôt que d’inférer le routage depuis les noms de runs. Pour chacune des 18 couches, la table primaire de 32 000 entrées est exactement une permutation par couche de $t \bmod 4$, soit 8 000 IDs primaires par expert. La table secondaire est distincte du primaire et suit la construction déterministe équilibrée de l’équation 2. Le run ne contient aucune table de fréquences ; il ne constitue donc pas une évaluation du bin-packing Zipf.

7 Discussion

7.1 Comparaison avec les architectures existantes

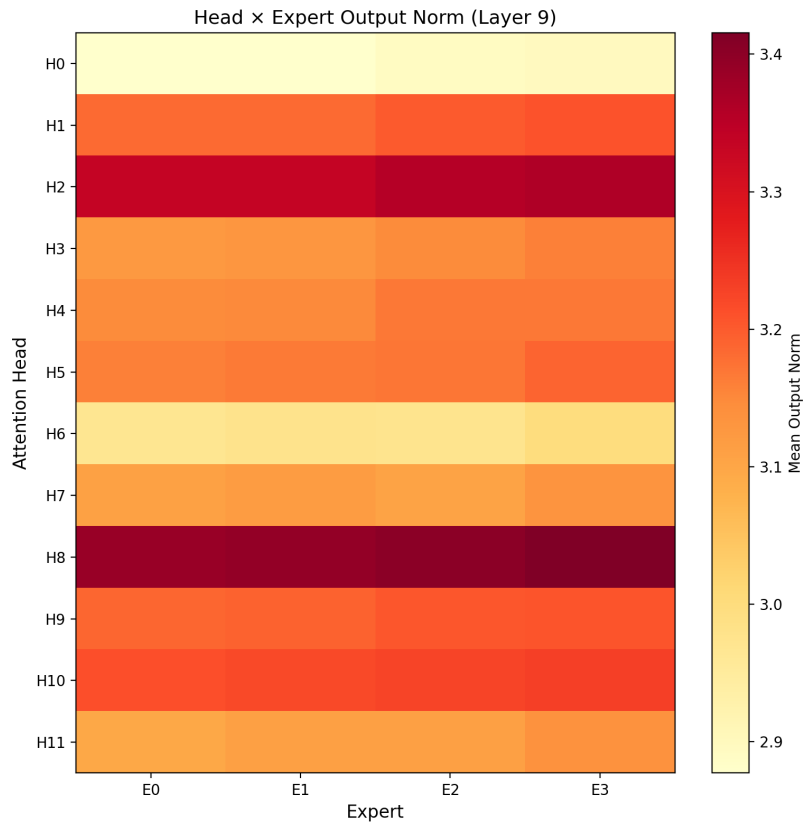


Figure 3: Heatmap diagnostique tête d’attention $\times$ expert pour la couche médiane. La figure est descriptive ; aucun couplage tête-expert n’en est inféré.

Table 3: Comparaison architecturale

Architecture	Routage	Perte d'équilibre
Transformer	Non	N/A
Switch Transformer	Souple	Requise
Mixtral	Top-2	Requise
COMPLEXITY-DEEP	Lexical dur	Non utilisée

7.2 Analyse du routage par token

Les diagnostics du run 300M mesurent le trafic réalisé sans attribuer ce trafic à un mécanisme frequency-aware :

- **Utilisation proche de l'équilibre** : dans le run 300M, l'utilisation finale observée des experts est 0,2481/0,2635/0,2481/0,2403, sans expert mort.
- **Normes équilibrées** : les normes des experts restent proches entre branches routées.
- **Aucune branche morte** : les quatre experts reçoivent un trafic observé non nul

Ces diagnostics excluent des branches routées mortes dans le run observé, mais n'établissent aucune spécialisation sémantique ou fonctionnelle.

Table 4: Token-Routed MLP vs MoE à routage souple

Critère	Token-Routed	MoE standard	Avantage
Table de sélection	Lexicale fixe	Apprise, dépendante du contenu	Dépend
Claim de spécialisation	Non établi ici	Variable selon le modèle	Aucun
Déploiement	Sans réseau de gating	Gating + dispatch	Token-Routed
Déterminisme du lookup	Oui	Possible à l'inférence	Dépend
Branche partagée	Oui	Optionnelle	Dépend
Signal de sélection	Identité fixe du token	Contenu de l'état caché	MoE standard

7.3 Substrat computationnel dense pour modules conditionnels

L'architecture évaluée peut être interprétée comme un cas particulier d'un principe d'intégration dense-first. Soit F_{dense} une transformation appliquée uniformément à tous les tokens et $G_{\text{conditional}}$ un module structuré dont les paramètres ou le chemin d'exécution dépendent d'une condition discrète z :

$$\mathbf{y} = F_{\text{dense}}(\mathbf{x}; \theta_s) + \alpha G_{\text{conditional}}(\mathbf{x}, z; \theta_c). \quad (12)$$

Le chemin dense fournit un calcul batché régulier exprimable par des multiplications matricielles standard. Le chemin conditionnel peut rester un résidu étroit, limitant la fraction de la couche affectée par le dispatch irrégulier, les tailles de groupes variables et les petits kernels experts. Dans COMPLEXITY-DEEP, F_{dense} est la branche SwiGLU partagée, z est l'identité du token et $G_{\text{conditional}}$ est le résidu lexical top-2 déterministe.

Cette décomposition ne rend pas des objets Python arbitraires directement exécutables par PyTorch. Un module conditionnel doit toujours être représenté par des tensors, des paramètres enregistrés et des opérations tensorielles différentiables ; une exécution efficace peut en outre nécessiter des multiplications matricielles groupées ou des kernels spécialisés. Le chemin partagé préserve plutôt un large substrat computationnel régulier tandis que le composant conditionnel reste étroit et structuré. Nos expériences valident uniquement l'instance de routage lexical. Elles ne démontrent ni que des modules conditionnels arbitraires bénéficient de la même décomposition, ni un gain wall-clock ; le débit inférieur de l'implémentation routée montre que l'overhead du dispatch sparse reste significatif.

7.4 Ablations exploratoires 100M (1B tokens)

Nous conservons la suite demandée de sept runs comme diagnostic exploratoire. Chaque run utilise le même budget sur $2 \times B200$: $954 \text{ steps} \times 2 \text{ GPUs} \times \text{batch/GPU } 256 \times \text{longueur } 2048 = 1,0003\text{B tokens}$. Le chemin FineWeb streaming n’a pas rempli la table de fréquences attendue par la configuration auparavant nommée “Zipf”. Cette condition a donc utilisé un routage primaire modulo avec une route secondaire déterministe ; nous la renommons *top-2 modulo-primaire/secondaire équilibré*. Les contrôles modulo et round-robin ont des assignations primaires équivalentes dans ce cadre : leurs différences single-run ne doivent pas être interprétées causalement.

Le run modulo-primaire/secondaire équilibré avec branche partagée obtient les plus faibles pertes loggées : 4,8205 en train au step 950 et 4,8887 en évaluation au step 750, contre 4,8244 et 4,8958 pour dense. Ces écarts sont faibles, à graine unique et comparables aux différences entre contrôles nominalelement équivalents. La suite fournit uniquement une évidence exploratoire ; elle n’établit aucun effet causal de stratégie ni bin-packing sensible aux fréquences.

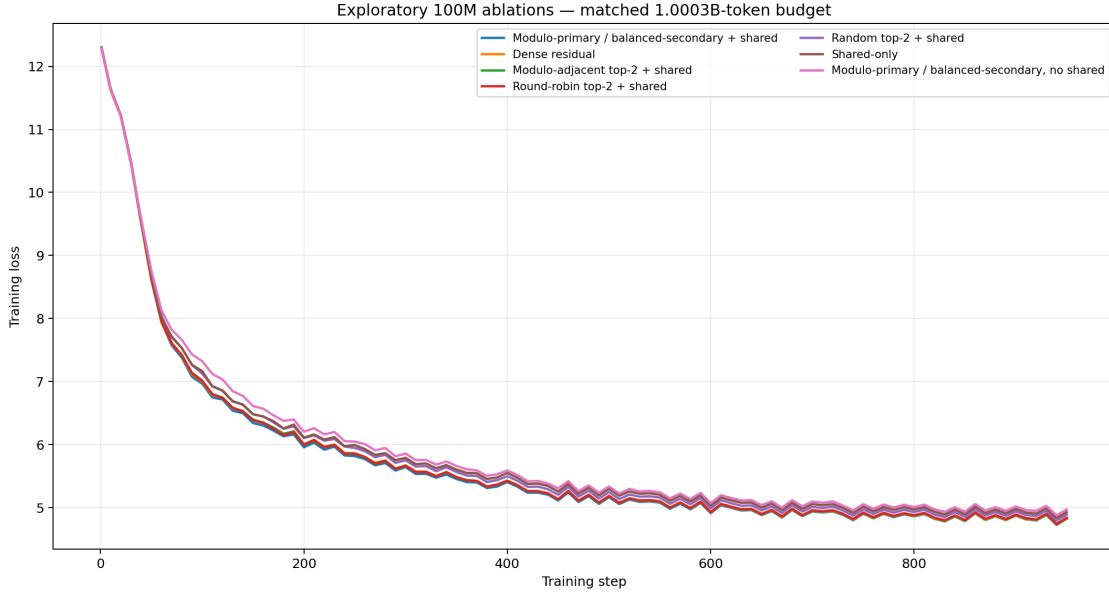


Figure 4: Courbes train exploratoires 100M à budget apparié. La condition auparavant nommée “Zipf shared” est identifiée par son lookup réel : top-2 modulo-primaire/secondaire équilibré.

Table 5: Runs exploratoires 100M à graine unique sur $2 \times B200$. Toutes les lignes utilisent 1,0003B tokens ; les valeurs d’évaluation sur le flux fixe issu du split train sont reportées au step 750.

Variante	Train loss @ 950	Meilleure val. @ 750	Tok/s fin
Modulo-primaire/secondaire équilibré + shared	4,8205	4,8887	321k
Dense residual	4,8244	4,8958	397k
Modulo-adjacent top-2 + shared	4,8320	4,9016	321k
Round-robin top-2 + shared	4,8384	4,9072	321k
Random top-2 + shared	4,8875	4,9577	318k
Shared-only	4,9285	5,0012	402k
Modulo-primaire/secondaire équilibré, sans shared	4,9693	5,0393	334k

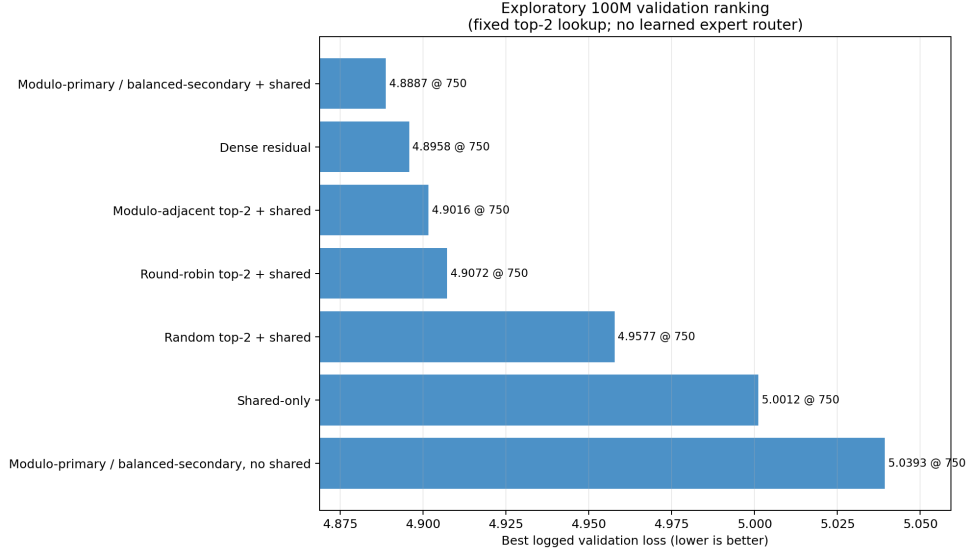


Figure 5: Pertes d’évaluation exploratoires 100M. Toutes les variantes routées utilisent des tables top-2 fixes ; aucune n’utilise de routeur d’experts appris.

7.5 Comparaison de scaling (300M, 8B tokens)

Pour valider l’architecture corrigée à plus grande échelle, nous entraînons des modèles iso-paramètres (Table 2) sur un budget FineWeb-Edu de 8B tokens. Les deux runs utilisent le même batch global de 1 048 576 tokens/step (8 GPUs $\times$ batch 64 $\times$ séquence 2048) ; numéro de step identique équivaut donc à tokens vus identiques, sans interpolation nécessaire.

Les deux runs partent d’une perte initiale comparable (TR 10,58, dense 10,54). Token-Routed est en retard durant la phase initiale : au step 100 ($\approx$ 105M tokens), l’écart est de +0,177 en faveur de dense, avec un pic à +0,31 vers le step 40. Le premier gain train loggé apparaît au step 740 et le premier gain sur le flux d’évaluation au step 750. Au dernier checkpoint d’évaluation commun (step 7 500 ; 7,864B tokens), Token-Routed atteint **2,932897** contre **2,948246**. Au step 7 620, la perte train est **2,923100** contre **2,932437** ; la moyenne des 50 derniers steps uniques vaut **2,928258** contre **2,944597**. Le flux d’évaluation provient du split train FineWeb-Edu et n’est pas un holdout indépendant.

Table 6: Comparaison scaling 300M à iso-batch (1 048 576 tokens/step), run complet 8B tokens. Train-loss au même step. Un écart négatif indique Token-Routed en avance. Dernière ligne : moyenne lissée sur les 50 derniers steps logés.

Step	Tokens vus	Dense (306,5M)	TR (306,5M, k=2)	Écart (TR – Dense)
100	105M	6,6698	6,8464	+0,177
500	524M	4,4450	4,4796	+0,035
700	734M	3,8567	3,8619	+0,005
1000	1,05B	3,5500	3,5324	−0,018
2000	2,10B	3,1953	3,1720	−0,023
4000	4,19B	3,0925	3,0738	−0,019
6000	6,29B	3,0061	2,9906	−0,015
7620	7,99B	2,9324	2,9231	−0,009
lissé 50 derniers	—	2,9446	2,9283	−0,016

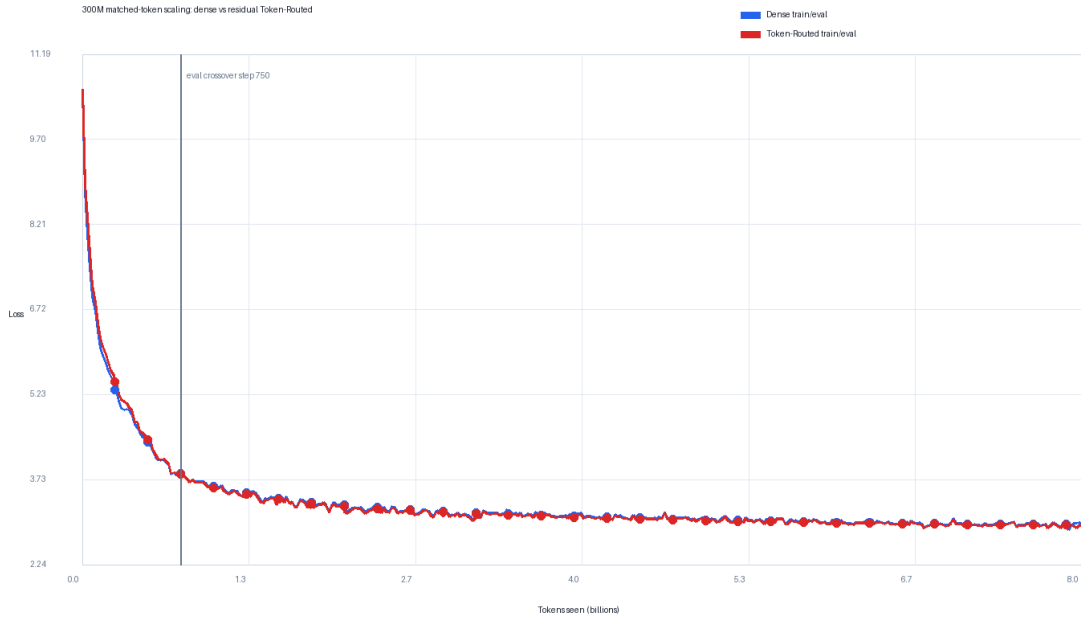


Figure 6: Courbes de scaling 300M corrigées à iso-batch (1 048 576 tokens/step). Token-Routed (rouge) est en retard durant les ≈ 700 premiers steps, atteint la parité, puis passe devant.

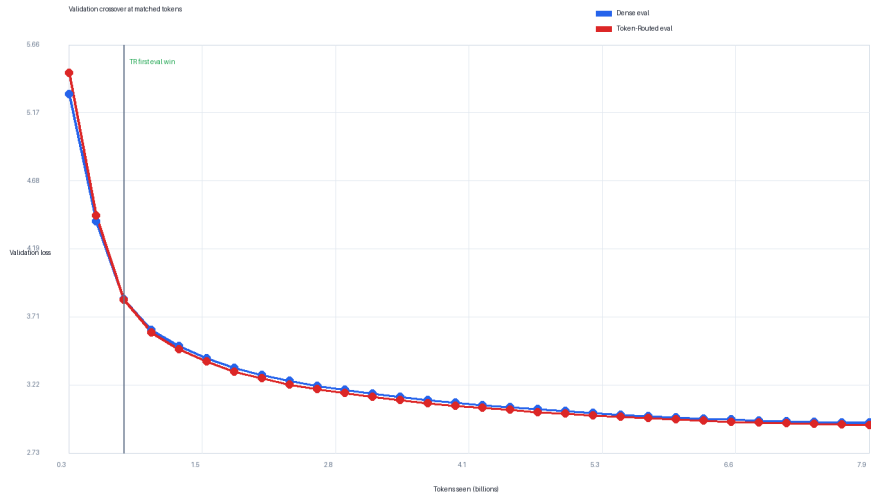


Figure 7: Loss du flux d'évaluation au même step. Token-Routed présente un déficit initial, croise dense au step 750 et reste devant jusqu'au dernier checkpoint commun au step 7 500 (7,864B tokens). Le flux provient du split train.

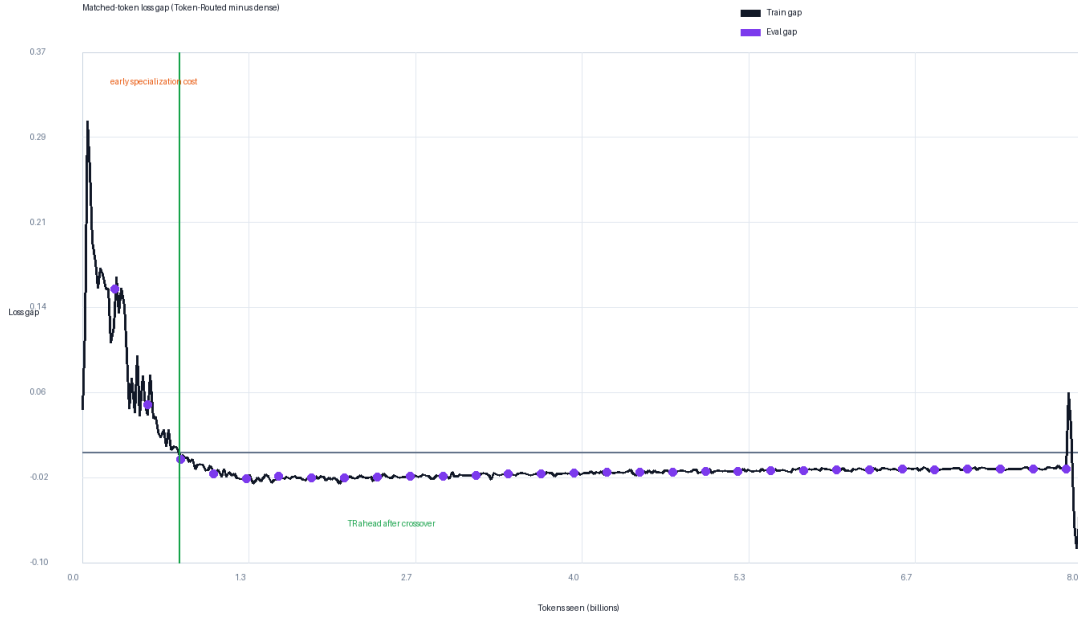


Figure 8: Écart de perte d’entraînement au même step (= tokens vus, iso-batch), calculé comme perte Token-Routed moins perte dense. Les valeurs négatives indiquent que Token-Routed est devant ; l’écart devient négatif au step loggé 740.

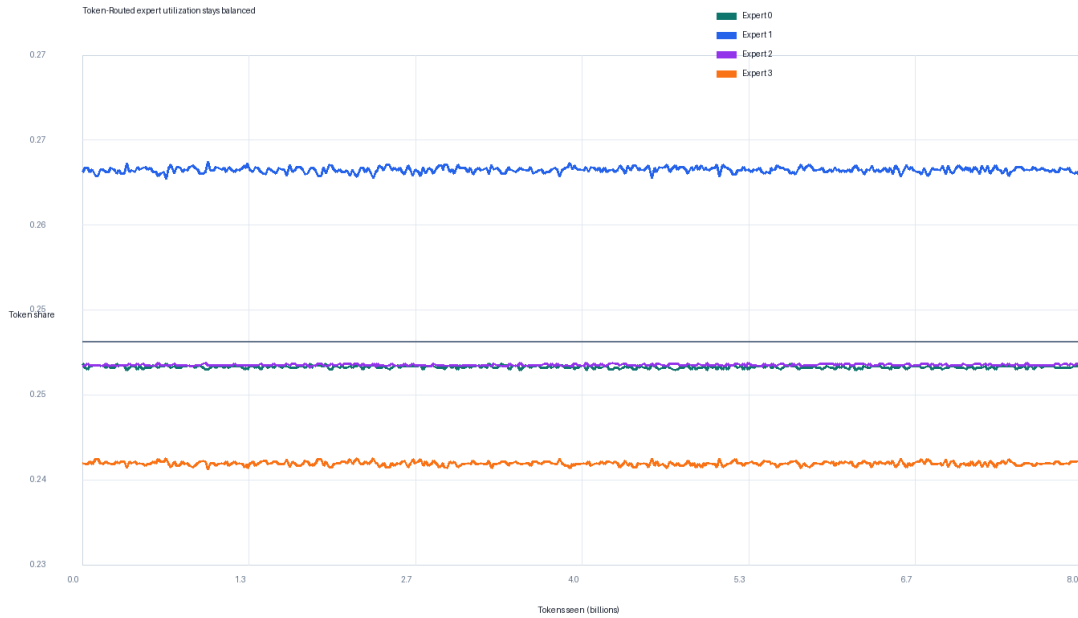


Figure 9: Utilisation des experts pendant le run Token-Routed 300M corrigé. Les quatre experts restent actifs et proches de l’équilibre ; le gain observé ne vient pas d’un effondrement d’expert.

Throughput d’entraînement. Le modèle Token-Routed 300M utilise une grande branche partagée plus une petite branche routée $\text{top-}k = 2$; il ne vise donc pas à être un benchmark de vitesse $\text{top-}1$ pur. Les deux runs ont été entraînés sur $8 \times \text{B300}$; dense atteint environ 0,95M tokens/s et Token-Routed environ 0,75M tokens/s au même batch global (1 048 576 tokens/step). Toutes les comparaisons de qualité ci-dessus sont à tokens vus appariés. Une comparaison wall-clock appariée répondrait à une autre question—si l’implémentation actuelle est plus efficace en calcul—et favoriserait la baseline dense plus rapide tant que la branche résiduelle routée n’est pas davantage optimisée. Nous rapportons donc le throughput séparément et ne revendiquons pas d’efficacité wall-clock. Le lookup de la table de routage est lui-même $O(1)$ et ne nécessite pas de synchronisation de routeur appris.

7.6 Vérification d’inférence

Pour vérifier que le routage lexical déterministe est compatible avec les stacks de serving standards, nous benchmarkons également un petit checkpoint Token-Routed antérieur sur vLLM 0.18 avec PagedAttention et CUDA graphs. Ce résultat de serving n’est pas utilisé comme preuve pour la comparaison de scaling 300M corrigée ci-dessus, et ne doit pas être interprété comme un claim d’efficacité compute pour le checkpoint 300M. Il vérifie seulement qu’un routage déterministe par token peut être déployé sans routeur appris ni dispatch all-to-all.

Sur un seul GPU NVIDIA RTX PRO 6000 (96 Go), avec 100 requêtes concurrentes à 10 RPS, 128 tokens d’entrée et 1900 tokens de sortie, le checkpoint atteint un débit soutenu de **8 078 tokens/s** (pic 10 179 tokens/s), avec un TTFT médian de 29,3ms et une ITL médiane de 7,9ms (Figure 10). Les mesures d’inférence mises à jour pour le checkpoint 300M corrigé restent un travail futur.

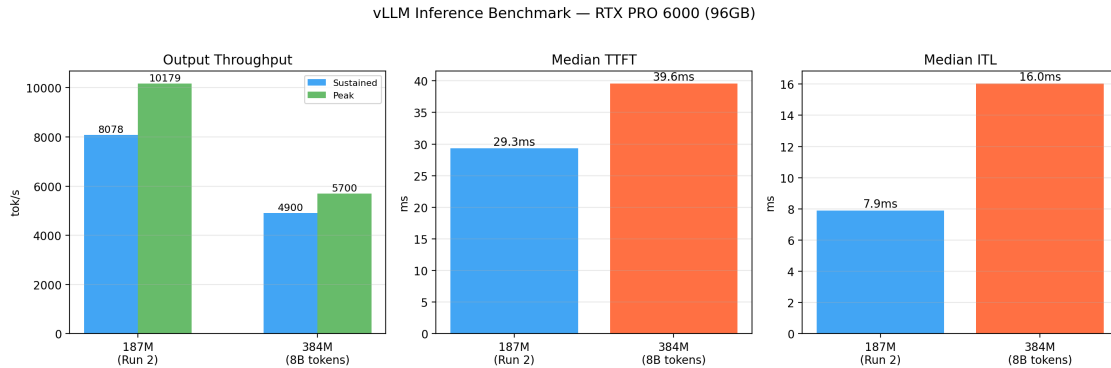


Figure 10: Vérification d’inférence sur un seul NVIDIA RTX PRO 6000 (96 Go). 100 requêtes à 10 RPS, 128 tokens d’entrée, 1900 tokens de sortie. Débit soutenu : 8 078 tok/s ; pic : 10 179 tok/s ; TTFT médian : 29,3ms ; ITL médiane : 7,9ms. Ce benchmark utilise un petit checkpoint antérieur et n’est rapporté que comme vérification de compatibilité de déploiement.

7.7 Limitations et travaux futurs

- **Résultat de scaling single-seed** : le run 300M corrigé est une comparaison complète à budget aligné sur 8B tokens, mais il reste un seul seed par architecture. Des seeds supplémentaires quantifieraient la robustesse de l’écart lissé final de $-0,0163$ à la variance d’initialisation.
- **Absence de résultat frequency-aware** : les checkpoints évalués utilisent des tables lexicales modulo permutées. Le bin-packing sensible aux fréquences existe comme chemin de code non évalué et n’est pas soutenu par les résultats rapportés.
- **Bin-packing Zipf en travaux futurs** : une étude propre devra estimer les fréquences sur le flux d’entraînement exact, construire et sérialiser la table gloutonne avant l’initialisation, vérifier la charge attendue et réalisée, puis comparer à la baseline modulo permutée sur plusieurs seeds. Les

checkpoints devront stocker un hash de la table et la provenance des fréquences afin d’empêcher tout fallback silencieux.

- **Benchmarks standardisés** : l’évaluation zero-shot sur ARC-Easy, HellaSwag et MMLU via `lm-evaluation-harness` doit être relancée sur les checkpoints 300M corrigés.

8 Conclusion

Nous avons présenté COMPLEXITY-DEEP, une architecture qui ajoute à un tronc MLP dense partagé une petite branche lexicale résiduelle top-2 déterministe, sans routeur appris ni perte auxiliaire d’équilibrage.

Dans le run 300M corrigé, la variante Token-Routed présente un déficit initial, gagne d’abord sur la perte train au step 740 et sur le flux d’évaluation au step 750. Au dernier checkpoint d’évaluation commun, elle reste devant ; l’écart final sur la moyenne des 50 derniers steps uniques est $-0,0163$. Ce résultat est un signal de qualité à tokens appariés pour cette paire de runs, sans preuve de généralisation multi-seed, de holdout indépendant ou d’avantage wall-clock.

Déclaration d’impact sociétal

Ce travail étudie une modification architecturale pour les modèles de langage. La soumission n’inclut pas les poids ; toute publication future devra respecter les exigences applicables de données, de sécurité et de licence.

Déclaration de reproductibilité

Le supplément fournit une implémentation PyTorch corrigée par audit, des tests exacts du routage, les configurations, les scripts de figures et le tokenizer 32k vérifié. Il s’agit d’un artefact de reproduction, non d’un snapshot historique inchangé.

References

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901, 2020.
- Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.
- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pp. 16344–16359, 2022.
- Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. Scaling vision transformers to 22 billion parameters. In *International Conference on Machine Learning*, pp. 7480–7512, 2023.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research*, 23(1):5232–5270, 2022.
- Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Huy Nguyen, Thong T. Doan, Quang Pham, Nghi D. Q. Bui, Nhat Ho, and Alessandro Rinaldo. On deepseekmoe: Statistical benefits of shared experts and normalized sigmoid gating. *arXiv preprint arXiv:2505.10860*, 2025.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.
- Stephen Roller, Sainbayar Sukhbaatar, Arthur Szlam, and Jason Weston. Hash layers for large sparse models. *Advances in Neural Information Processing Systems*, 34:17555–17566, 2021.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarsz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

A Courbes d’entraînement

Des courbes d’entraînement et figures d’analyse supplémentaires sont disponibles dans le matériel supplémentaire.